Lab_Assignment_3.2

AI Assisted Coding B37 2303A52077

Task Description-1

Progressive Prompting for Calculator Design: Ask the AI to design a simple calculator program by initially providing only the function name. Gradually enhance the prompt by adding comments and usage examples

#Initial Code:

first.py third.py

third.py > ...

```
#write a python code AI to design a simple calculator program by initially providing only the function name
'''
def calculator():
    def add(x, y):
        return x + y

    def subtract(x, y):
        return x - y

    def multiply(x, y):
        return x * y

    def divide(x, y):
        if y == 0:
            return "Error! Division by zero."
        return x / y

    print("Select operation:")
    print("1. Add (+)")
    print("2. Subtract (-)")
    print("3. Multiply (*)")
    print("4. Divide (/)")

    choice = input("Enter choice (+|-|*|/): ")

    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    if choice == '+':
        print(f"{num1} + {num2} = {add(num1, num2)}")
    elif choice == '-':
        print(f"{num1} - {num2} = {subtract(num1, num2)}")
    elif choice == '*':
        print(f"{num1} * {num2} = {multiply(num1, num2)}")
    elif choice == '/':
        print(f"{num1} / {num2} = {divide(num1, num2)}")
    else:
        print("Invalid input")

calculator()
'''
```

#Final code

```
import math
import os
def clear_screen():
    os.system('cls' if os.name == 'nt' else 'clear')
def get_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Invalid input! Please enter a numeric value.")
def calculator():
    history = []
    operations = {
        '1': ("Add (+)", lambda x, y: x + y),
        '2': ("Subtract (-)", lambda x, y: x - y),
        '3': ("Multiply (*)", lambda x, y: x * y),
        '4': ("Divide (/)", lambda x, y: "Error! Division by zero." if y ==
0 else x / y),
        '5': ("Modulus (%)", lambda x, y: x % y),
        '6': ("Exponentiation (^)", lambda x, y: x ** y),
        '7': ("Floor Division (//)", lambda x, y: "Error! Division by zero."
if y == 0 else x // y),
        '8': ("Square Root (√)", lambda x: math.sqrt(x)),
        '9': ("Cubic Root (∛)", lambda x: x ** (1/3)),
        '10': ("Percentage (%)", lambda x, total: (x / total) * 100),
```

```
        '11': ("Average", lambda nums: sum(nums) / len(nums)),
        '12': ("Logarithm (log)", lambda x, base=10: math.log(x, base)),
        '13': ("Sine (sin)", lambda x: math.sin(math.radians(x))),
        '14': ("Cosine (cos)", lambda x: math.cos(math.radians(x))),
        '15': ("Tangent (tan)", lambda x: math.tan(math.radians(x))),
        '16': ("Natural Log (ln)", lambda x: math.log(x)),
        '17': ("Exponential (e^x)", lambda x: math.exp(x)),
        '18': ("Factorial (!)", lambda x: "Error! Factorial of negative number doesn't exist." if x < 0 else math.factorial(int(x))),
        '19': ("Combination (nCr)", lambda n, r: "Error! r cannot be greater than n." if r > n else math.comb(int(n), int(r))),
        '20': ("Permutation (nPr)", lambda n, r: "Error! r cannot be greater than n." if r > n else math.perm(int(n), int(r))),
        '21': ("GCD", lambda x, y: math.gcd(int(x), int(y))),
        '22': ("LCM", lambda x, y: abs(int(x) * int(y)) // math.gcd(int(x), int(y))),
    }
    while True:
        clear_screen()
        print("Select operation:")
        for key, (name, _) in operations.items():
            print(f"{key}. {name}")
        print("23. View History")
        print("24. Exit")

        choice = input("Enter choice: ")
```

```
        if choice == '24':
            print("Exiting the calculator. Goodbye!")
            break
        elif choice == '23':
            clear_screen()
            print("Calculation History:")
            for record in history:
                print(record)
            input("Press Enter to continue...")
            continue
        elif choice in operations:
            operation_name, operation_func = operations[choice]
            try:
                if choice in ['1', '2', '3', '4', '5', '6', '7']:
                    num1 = get_number("Enter first number: ")
                    num2 = get_number("Enter second number: ")
                    result = operation_func(num1, num2)
                    record = f"{operation_name}: {num1} and {num2} =
{result}"
                elif choice in ['8', '9']:
                    num = get_number("Enter number: ")
                    result = operation_func(num)
                    record = f"{operation_name}: {num} = {result}"
                elif choice == '10':
```

```
                part = get_number("Enter part value: ")
                total = get_number("Enter total value: ")
                result = operation_func(part, total)
                record = f"{operation_name}: {part} is {result}% of
{total}"
            elif choice == '11':
                nums = list(map(float, input("Enter numbers separated by
space: ").split()))
                result = operation_func(nums)
                record = f"{operation_name}: Average of {nums} =
{result}"
            elif choice == '12':
                num = get_number("Enter number: ")
                base_input = input("Enter base (default 10): ")
                base = float(base_input) if base_input else 10
                result = operation_func(num, base)
                record = f"{operation_name}: log_{base}({num}) =
{result}"
            elif choice in ['13', '14', '15']:
                angle = get_number("Enter angle in degrees: ")
                result = operation_func(angle)
                record = f"{operation_name}:
{operation_name.split()[0]}({angle}) = {result}"
            elif choice == '16':
                num = get_number("Enter number: ")
                result = operation_func(num)
```

```
                record = f"{operation_name}: ln({num}) = {result}"
            elif choice == '17':
                num = get_number("Enter number: ")
                result = operation_func(num)
                record = f"{operation_name}: e^{num} = {result}"
            elif choice == '18':
                num = get_number("Enter non-negative integer: ")
                result = operation_func(num)
                record = f"{operation_name}: {int(num)}! = {result}"
            elif choice in ['19', '20']:
                n = get_number("Enter n: ")
                r = get_number("Enter r: ")
                result = operation_func(n, r)
                record = f"{operation_name}: {int(n)}{operation_name[-
3:]}{int(r)} = {result}"
            elif choice in ['21', '22']:
                num1 = get_number("Enter first integer: ")
                num2 = get_number("Enter second integer: ")
                result = operation_func(num1, num2)
                record = f"{operation_name}:
{operation_name}({int(num1)}, {int(num2)}) = {result}"
            print(record)
            history.append(record)
        except Exception as e:
            print(f"An error occurred: {e}")
```

```
            input("Press Enter to continue...")
        else:
            print("Invalid input")
            input("Press Enter to continue...")
calculator()
```

Prompts that were used in enhance the prompt by adding comments and usage examples

1st Prompt:

"#write a python code AI to design a simple calculator program by initially providing only the function name"

2nd Prompt:"

#update the code to handle invalid inputs gracefully like non-numeric values and unsupported operations, and add a loop to allow multiple calculations until the user decides to exit.

#also do exception handling for division by zero, and provide an option to clear the screen before each calculation.

#add a other operations used in simple calculator like modulus, exponentiation, floor division, square & cubic rooots, percentages, average of numbers, log funtions, trigonometric functions & calculus calculations.

"

Final Prompt:"

#now update the above code to include a history feature that logs all calculations performed during the session and provides an option to view this history.

#aswell optimize the code for better readability and maintainability

"

Output:

```
Select operation:
1. Add (+)
2. Subtract (-)
3. Multiply (*)
4. Divide (/)
5. Modulus (%)
6. Exponentiation (^)
7. Floor Division (//)
8. Square Root (√)
9. Cubic Root (∛)
10. Percentage (%)
11. Average
12. Logarithm (log)
13. Sine (sin)
14. Cosine (cos)
15. Tangent (tan)
16. Natural Log (ln)
17. Exponential (e^x)
18. Factorial (!)
19. Combination (nCr)
20. Permutation (nPr)
21. GCD
22. LCM
23. View History
24. Exit
Enter choice:
```

```
Calculation History:
Add (+): 10.0 and 9.0 = 19.0
Subtract (-): 10.0 and 9.0 = 1.0
Multiply (*): 10.0 and 9.0 = 90.0
Divide (/): 10.0 and 9.0 = 1.1111111111111112
Modulus (%): 10.0 and 9.0 = 1.0
Exponentiation (^): 10.0 and 9.0 = 1000000000.0
Floor Division (//): 10.0 and 9.0 = 1.0
Press Enter to continue...
```

Task Description-2

- Refining Prompts for Sorting Logic: Start with a vague prompt for sorting student marks, then refine it to clearly specify sorting order and constraints.

vague prompt for sorting student marks

Prompt: #write a python program that sort the given student marks

```
365  #write a python program that sort the given student marks
366
367  def sort_student_marks(marks):
368      return sorted(marks)
369
370  marks = [88, 56, 90, 67, 45, 78]
371  sorted_marks = sort_student_marks(marks)
372  print("Sorted student marks:", sorted_marks)
373
```

INPUT: [88, 92, 79, 85, 95, 67, 73, 90]

OUTPUT: Sorted student marks: [67, 73, 79, 85, 88, 90, 92, 95]

Prompts used to refine the code:

#write a python program that sort the given student marks

#update the code to sort in descending order

#update the code to user input for no of students and their marks and there names aswell

#and add feature to search for a particular student's marks by their name

#add exception handling for invalid inputs 0 < marks > 100

#and as per there marks assign grades A, B, C, D, F

#store the data in a dictionary with student names as keys and a tuple of (names, marks, grade) as values

#then sort the dictionary based on marks in descending order and print the sorted list of students with their marks and grades

#finally optimize the code for better readability and maintainability

first.py third.py

third.py > ...

```python
#write a python program that sort the given student marks
#update the code to sort in descending order
#update the code to user input for no of students and their marks and there names aswell
#and add feature to search for a particular student's marks by their name
#add exception handling for invalid inputs 0 < marks > 100
#and as per there marks assign grades A, B, C, D, F
#store the data in a dictionary with student names as keys and a tuple of (names, marks, grade) as values
#then sort the dictionary based on marks in descending order and print the sorted list of students with their marks and grades
#finally optimize the code for better readability and maintainability

def get_grade(marks):
    if 90 <= marks <= 100:
        return 'A'
    elif 80 <= marks < 90:
        return 'B'
    elif 70 <= marks < 80:
        return 'C'
    elif 60 <= marks < 70:
        return 'D'
    elif 0 <= marks < 60:
        return 'F'
    else:
        return None
def main():
    student_data = {}
    try:
        num_students = int(input("Enter the number of students: "))
        for _ in range(num_students):
            name = input("Enter student name: ")
            marks = float(input(f"Enter marks for {name} (0-100): "))
            if marks < 0 or marks > 100:
                raise ValueError("Marks must be between 0 and 100.")
            grade = get_grade(marks)
            student_data[name] = (name, marks, grade)

        # Sort the dictionary based on marks in descending order
        sorted_students = dict(sorted(student_data.items(), key=lambda item: item[1][1], reverse=True))

        print("\nSorted Student Marks and Grades:")
        for name, (student_name, marks, grade) in sorted_students.items():
            print(f"Name: {student_name}, Marks: {marks}, Grade: {grade}")
```

```
406
407             # Search for a particular student's marks by their name
408             search_name = input("\nEnter a student's name to search for their marks: ")
409             if search_name in student_data:
410                 student_name, marks, grade = student_data[search_name]
411                 print(f"Found - Name: {student_name}, Marks: {marks}, Grade: {grade}")
412             else:
413                 print("Student not found.")
414         except ValueError as ve:
415             print(f"Invalid input: {ve}")
416         except Exception as e:
417             print(f"An error occurred: {e}")
418 if __name__ == "__main__":
419     main()
420
```

```
PS D:\Subjects\ai assited coding\ai_assi> & C:/Users/ASUS/AppData/
Enter the number of students: 10
Enter student name: vishal
Enter marks for vishal (0-100): 100
Enter student name: isha
Enter marks for isha (0-100): 90
Enter student name: radha
Enter marks for radha (0-100): 45
Enter student name: sneha
Enter marks for sneha (0-100): 59
Enter student name: mahesh
Enter marks for mahesh (0-100): 80
Enter student name: meena
Enter marks for meena (0-100): 76
Enter student name: varun
Enter marks for varun (0-100): 97
Enter student name: yash
Enter marks for yash (0-100): 23
Enter student name: uma
Enter marks for uma (0-100): 87
Enter student name: david
Enter marks for david (0-100): 78

Sorted Student Marks and Grades:
Name: vishal, Marks: 100.0, Grade: A
Name: varun, Marks: 97.0, Grade: A
Name: isha, Marks: 90.0, Grade: A
Name: uma, Marks: 87.0, Grade: B
Name: mahesh, Marks: 80.0, Grade: B
Name: david, Marks: 78.0, Grade: C
Name: meena, Marks: 76.0, Grade: C
Name: sneha, Marks: 59.0, Grade: F
Name: radha, Marks: 45.0, Grade: F
Name: yash, Marks: 23.0, Grade: F

Enter a student's name to search for their marks: isha
Found - Name: isha, Marks: 90.0, Grade: A
PS D:\Subjects\ai assited coding\ai_assi>
```

Task Description-3

- Few-Shot Prompting for Prime Number Validation: Provide multiple input-output examples for a function that checks whether a number is prime. Observe how few-shot prompting improves correctness.
- PROMPT: “#write a python code to check if a given number is prime or not”

#Code:

```
422  #write a python code to check if a given number is prime or not
423
424  def is_prime(n):
425      if n <= 1:
426          return False
427      for i in range(2, int(n**0.5) + 1):
428          if n % i == 0:
429              return False
430      return True
431  number = int(input("Enter a number: "))
432  if is_prime(number):
433      print(f"{number} is a prime number.")
434  else:
435      print(f"{number} is not a prime number.")
```

#ip/op

```
Enter a number: 45
45 is not a prime number.
```

Task Description-4

- Prompt-Guided UI Design for Student Grading System: Create a user interface for a student grading system that calculates total marks, percentage, and grade based on user input.
- Prompt: “

```
#write a python code for a student to calculate their total marks,
average marks,
```

- `# and grade based on their scores for the no of subjects as per the user input and marking scheme`

“

```python
#write a python code for a student to calculate their total marks, average marks,
# and grade based on their scores for the no of subjects as per the user input and marking scheme

def get_grade(average):
    if average >= 90:
        return 'A'
    elif average >= 80:
        return 'B'
    elif average >= 70:
        return 'C'
    elif average >= 60:
        return 'D'
    else:
        return 'F'
def main():
    try:
        num_subjects = int(input("Enter the number of subjects: "))
        total_marks = 0
        for i in range(num_subjects):
            marks = float(input(f"Enter marks for subject {i + 1}: "))
            if marks < 0 or marks > 100:
                raise ValueError("Marks must be between 0 and 100.")
            total_marks += marks
        average_marks = total_marks / num_subjects
        grade = get_grade(average_marks)
        print(f"\nTotal Marks: {total_marks}")
        print(f"Average Marks: {average_marks}")
        print(f"Grade: {grade}")
    except ValueError as ve:
        print(f"Invalid input: {ve}")
    except Exception as e:
        print(f"An error occurred: {e}")
if __name__ == "__main__":
    main()
```

```
Enter the number of subjects: 6
Enter marks for subject 1: 23
Enter marks for subject 2: 45
Enter marks for subject 3: 67
Enter marks for subject 4: 89
Enter marks for subject 5: 90
Enter marks for subject 6: 100

Total Marks: 414.0
Average Marks: 69.0
Grade: D
PS D:\Subjects\ai assited coding\ai_assi>
```

Task Description-5

- Analyzing Prompt Specificity in Unit Conversion Functions: Improving a Unit Conversion Function (Kilometers to Miles and Miles to Kilometers) Using Clear Instructions.
- Prompt:

```
•
• #write a python code to perform unit conversions between kilometers and
  miles,
```

```python
#write a python code to perform unit conversions between kilometers and miles,
def km_to_miles(km):
    return km * 0.621371
def miles_to_km(miles):
    return miles / 0.621371
def main():
    try:
        choice = input("Choose conversion (1: km to miles, 2: miles to km): ")
        if choice == '1':
            km = float(input("Enter distance in kilometers: "))
            miles = km_to_miles(km)
            print(f"{km} kilometers is equal to {miles} miles.")
        elif choice == '2':
            miles = float(input("Enter distance in miles: "))
            km = miles_to_km(miles)
            print(f"{miles} miles is equal to {km} kilometers.")
        else:
            print("Invalid choice.")
    except ValueError:
        print("Invalid input! Please enter a numeric value.")
if __name__ == "__main__":
    main()
```

Improved Prompt:

#write a python code to perform unit conversions between kilometers and miles,

#update the code to implement all the exception handling for invalid inputs and provide user-friendly messages.

#also take all required constraints into consideration while writing the code.

```python
#write a python code to perform unit conversions between kilometers and miles,
#update the code to implement all the exception handling for invalid inputs and provide user-friendly messages.
#also take all required constraints into consideration while writing the code.

def km_to_miles(km):
    return km * 0.621371
def miles_to_km(miles):
    return miles / 0.621371
def main():
    try:
        print("Unit Conversion:")
        print("1. Kilometers to Miles")
        print("2. Miles to Kilometers")
        choice = input("Enter choice (1/2): ")
        if choice == '1':
            km = float(input("Enter distance in kilometers: "))
            if km < 0:
                raise ValueError("Distance cannot be negative.")
            miles = km_to_miles(km)
            print(f"{km} kilometers is equal to {miles} miles.")
        elif choice == '2':
            miles = float(input("Enter distance in miles: "))
            if miles < 0:
                raise ValueError("Distance cannot be negative.")
            km = miles_to_km(miles)
            print(f"{miles} miles is equal to {km} kilometers.")
        else:
            print("Invalid choice. Please select 1 or 2.")
    except ValueError as ve:
        print(f"Invalid input: {ve}")
    except Exception as e:
        print(f"An error occurred: {e}")
if __name__ == "__main__":
    main()
```

```
                                    > & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe
Unit Conversion:
1. Kilometers to Miles
2. Miles to Kilometers
Enter choice (1/2): 2
Enter distance in miles: -7
Invalid input: Distance cannot be negative.
PS D:\Subjects\ai assited coding\ai_assi> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe
Unit Conversion:
1. Kilometers to Miles
2. Miles to Kilometers
Enter choice (1/2): 1
Enter distance in kilometers: 10
10.0 kilometers is equal to 6.21371 miles.
PS D:\Subjects\ai assited coding\ai_assi>
```